

TOFD_AWI 新框架架构设计文档

4轮车体超声波检测控制系统 · .NET Framework 4.8 · 2026-05-26

一、架构总览

新旧对比		
	旧框架	新框架
入口	Program.cs → Frm_Main_C	Program_New.cs → Frm_Main_C_New
主窗体规模	8,712 行 (God Object)	244 行 (纯 UI)
全局状态	SysInfo.cs 7,096 行静态类	6 个独立状态类, 每个 <70 行
依赖方向	UI → 直接 new DLL 类	Program_New → Service → 接口 ← 适配器 → DLL
可测试性	不可测 (强耦合)	可注入 Mock 独立测试
配置管理	散落在各处的 INI 读取	SystemConfig.Load() 集中加载
换硬件	改遍所有窗体	只换一个适配器

核心设计思想：接口隔离 + 依赖注入 + 适配器模式

上层代码永远只看到接口 (ITofdHardware / IMotionController) , 不知道底下是 Tofd_DLL.CLTofd_BJ 还是 CanCmd 。将来换硬件或改为模拟测试, 只换适配器实现即可。

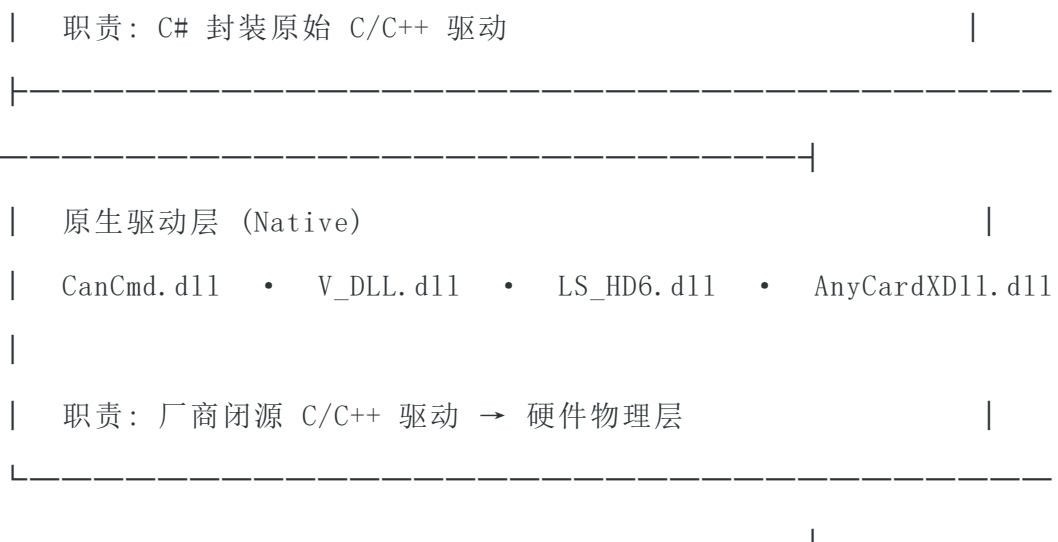
二、解决方案项目结构

项目	类型	职责	状态
----	----	----	----

Tofd_AWI	WinExe	主程序入口 + 窗体	改造中
ClassLibrary_Interface	Library	7 个硬件抽象接口 + 配置类	新增
NewInspect.Services	Library	服务层 + 适配器 + 状态类	新增
Tofd_DLL	Library	TOFD 超声采集卡封装 (3,737 行)	保持
Clb_MT_Comm	Library	CAN/COM 运动控制 (3,512 行)	保持
Clb_XmCam_DLL	Library	4 路工业相机 SDK (1,096 行)	保持
HD850_64	Library	激光轮廓仪驱动 (3,109 行)	保持
Frame_Work	Library	公共数据结构 (10,211 行)	保持
ClassLib_TestData	Library	测试数据实体	保持

三、分层架构设计





四、DI 组装流程

`Program_New.cs` 作为新入口点，按顺序组装整个应用：

```
static void Main()
{
    // Step 1 — 防止重复运行
    string processName = Process.GetCurrentProcess().ProcessName;
    if (Process.GetProcessesByName(processName).Length > 1) return;

    // Step 2 — 加载配置（替代 SysInfo 中散落的 INI 读取）
    var config = SystemConfig.Load();

    // Step 3 — 创建运行时状态（替代 SysInfo 的静态全局字段）
    var tofdState = new TofdState { Gain = 5f };
    var motionState = new MotionState { MarkEnabled = false };
    var systemState = new SystemRuntimeState { ... };
    ...

    // Step 4 — 创建硬件适配器（封装真实 DLL）
    ITofdHardware tofdHw = new TofdHardwareAdapter();
    IMotionController motionHw = new CanMotionControllerAdapter();

    // Step 5 — 创建服务层（注入接口 + 状态，替代窗体里的业务代码）
    var tofdService = new TofdService(tofdHw, null, tofdState, cScanState, config);
    var motionService = new MotionService(motionHw, motionState, config);
    var cameraService = new CameraService();
    var dataService = new DataService(config);

    // Step 6 — 启动新主窗体（注入所有服务）
    var mainForm = new Frm_Main_C_New(
        config, systemState,
        tofdService, motionService, cameraService, dataService,
        tofdState, motionState, projectState, reportState);
    Application.Run(mainForm);
}
```

对比旧入口 `Program.cs`：直接 `Application.Run(new Frm_Main_C())`，无任何注入。

五、接口定义

5.1 完整接口清单

所有接口定义在 `ClassLibrary_Interface/IHardwareInterfaces.cs` (142 行)：

接口	适配器实现	状态	说明
ITofdHardware	TofdHardwareAdapter	桩	TOFD 超声采集
ICScanHardware	未实现	未创建	C-Scan 扫描
IMotionController	CanMotionControllerAdapter	P0 完成	车体运动控制
ICameraDevice	CameraAdapter	桩	工业相机
ILaserProfiler	未实现	待规划	激光轮廓仪
IPowerMonitor	未实现	待规划	电池监控
IEddyCurrentDevice	未实现	待规划	涡流检测

5.2 核心接口方法 (IMotionController)

方法	CAN 指令	说明
Connect(port, isCom)	—	InitCan(125,5) / InitCom(port)
MoveForward()	SendData(1,1,6,0,"1")	帧 43 52 01 ...
MoveBackward()	SendData(1,1,6,0,"2")	帧 43 52 02 ...
Stop()	SendData(1,1,6,0,"3")	帧 43 52 03 ...
SetSpeed(%)	SendData(2,2,0,0,"pct,50")	手动速度参数
SetGratingArm(up)	SendData(5,1,0,0,"0"/"1")	0=提起 1=落下
MarkDefect()	SendData(5,2,0,0,"1")	缺陷打标
GetDistance()	回传帧解析	从编码器读取

六、完整调用链路

6.1 运动控制链路（P0 已实现）

用户点击 "前进" 按钮

```
→ Frm_Main_C_New.Btn_Forward_Click()
  → _motionService.Forward()
    → _controller.MoveForward()    ← 接口调用，不依赖具体类
      → CanMotionControllerAdapter.MoveForward()
        → _mtComm.SendData(1, 1, 6, 0, "1")
          → MT_Comm.SendData() - 拼帧头 → byte[]
            → MT_Comm.Can_Send(strSend) - 拆成 8 字节 CAN 帧
              → CanCmd.CAN_ChannelSend( ... )    ← [DllImport("CanCmd.dll")]
                → CanCmd.dll (原生 C 驱动) → USB-CAN 硬件 → CAN 总线 → 车体 MCU
```

6.2 心跳保活机制

适配器内部启动 `System.Threading.Timer`，每 1.5 秒发送 `SendData(1, 3, 0, 0, "99")`。
车体 MCU 约 3 秒收不到心跳即判定主机断线，自动停车。

6.3 TOFD 链路（桩，待实现）

用户点击 "开始检测"

```
→ _tofdService.StartAScan(callback)
  → _hardware.StartAcquisition(channel, callback)
    → TofdHardwareAdapter.StartAcquisition()    ← 目前空桩，需接入 Tofd_DLL
      → [DllImport] → AnyCardXDll.dll → 超声采集卡
```

七、系统状态管理

旧框架中所有状态散落在 SysInfo.cs （7,096 行）的 public static 字段中，全局任意位置可读写，极难追踪。

新框架将状态收敛为 6 个独立类（ NewInspect.Services/State/RuntimeState.cs ， 186 行）：

状态类	替代的 SysInfo 字段	关键属性
TofdState	m_SysBuff, m_fl_Gain, m_i_UI_Type	IsLinked, Gain, ExposureTime, ProbeType
CScanState	m_SysBuff_C, m_C_Item_Info	DataBuff, ItemInfo, WaitTimeMs
MotionState	Climb 相关全部字段	SpeedPercent, Direction, DistanceX/Y, GratingArmState
SystemRuntimeState	m_iLanguage, m_i_TOFD_0, m_bl_Save	Language, WorkMode, IsSaved, VideoRecording
InspectionProjectState	检测项目字段	WeldId, ItemName, IsNewItem
ReportState	报表缓存	PrintItems, ReportParams

设计原则：每个状态类只包含相关字段，不包含任何业务逻辑。状态通过构造函数注入，由服务层统一读写，窗体只做展示。

八、系统配置

SystemConfig.cs 集中管理所有 INI 配置，替代散落在各处的 IniReadDefine 调用。

分类	属性	INI Section/Key	默认值
基础	Language	System/m_iLanguage	0 (中文)
基础	WorkMode	System/m_i_TOFD_0_Cscan_1	0 (TOFD)
网络	NetworkIp	IP/IP	192.168.1.10
通讯	CommMode	COM_Can/m_blCom1_Can0	1 (COM)

通讯	CommPort	COM_Can/COM	COM3
运动	SpeedPercent	SYSInfo/iSpeed_Xs	50
运动	GratingSpeed	SYSInfo/i_Para_Speed_Gsb	50
TOFD	SoundVelocity	TOFD/SoundVelocity (HardConfig)	5900 m/s
TOFD	ProbeCenterDistance	TOFD/PCS (HardConfig)	50 mm
TOFD	ProbeAngle	TOFD/ProbeAngle (HardConfig)	60°

九、新主窗体

9.1 控件布局

1280 × 800 窗体，居中显示。控件通过 `Frm_Main_C_New.Designer.cs` 定义（VS 设计器原生格式，支持可视化编辑）：

名称	类型	位置	功能
Lb_LinkState	Label	10,10 / 150×25	显示联机状态
Lb_Distance	Label	200,10 / 150×25	显示行进距离
Lb_Speed	Label	400,10 / 100×25	显示当前速度
Btn_Link	Button	10,50 / 100×35	连接设备
Btn_Forward	Button	120,50 / 80×35	前进
Btn_Backward	Button	210,50 / 80×35	后退
Btn_Stop	Button	300,50 / 80×35	停止
Btn_Mark	Button	390,50 / 80×35	打标
Btn_StartScan	Button	500,50 / 100×35	开始检测
Btn_StopScan	Button	610,50 / 100×35	停止检测
Pic_AScan	PictureBox	10,100 / 600×300	A 扫描波形
Pic_BScan	PictureBox	620,100 / 300×300	B 扫描图像
Pic_DScan	PictureBox	10,410 / 600×150	D 扫描图像

9.2 按钮事件映射

按钮	事件	调用链
连接	Btn_Link_Click	_tofdService.Connect() + _motionService.Connect()
前进	Btn_Forward_Click	_motionService.Forward()
后退	Btn_Backward_Click	_motionService.Backward()
停止	Btn_Stop_Click	_motionService.Stop()
打标	Btn_Mark_Click	_motionService.MarkDefect()
开始检测	Btn_StartScan_Click	_tofdService.StartAScan(callback)
停止检测	Btn_StopScan_Click	_tofdService.StopAScan() + _motionService.Stop()

9.3 生命周期管理

OnFormClosing() 重写确保资源正确释放：停止所有扫描、断开硬件连接、释放相机资源。

十、硬件 CAN 协议参考

车体运动控制通过 `Clb_MT_Comm.MT_Comm.SendData(iComType_0, iComType_1, iDataType_0, iFloor_0, strData)` 发送 CAN 帧。

10.1 运动指令速查

操作	SendData 调用	帧结构
前进	<code>SendData(1, 1, 6, 0, "1")</code>	43 52 01 00 00 00 00 00
后退	<code>SendData(1, 1, 6, 0, "2")</code>	43 52 02 00 00 00 00 00
停止	<code>SendData(1, 1, 6, 0, "3")</code>	43 52 03 00 00 00 00 00
索取状态 (心跳)	<code>SendData(1, 3, 0, 0, "99")</code>	43 52 99 00 00 00 00 00
写速度	<code>SendData(2, 2, 0, 0, "speed,gsb")</code>	帧头 0x4A 0x56
光栅臂升降	<code>SendData(5, 1, 0, 0, up?"0":"1")</code>	0=提起 1=落下
扫描范围	<code>SendData(2, 1, 0, 0, "start,end")</code>	帧头 0x42 0x4C
纠偏控制	<code>SendData(1, 1, 8, 0, direction)</code>	1=左 2=右 3=停
打标	<code>SendData(5, 2, 0, 0, "1")</code>	触发缺陷标记
灯光	<code>SendData(4, 1, 3/4, 0, "50"/"0")</code>	前灯/后灯亮度

10.2 SendData 内部流程

`SendData(iComType_0, iComType_1, iDataType, iFloor, strData)`

- ① 根据参数拼帧头 → `byte[] _btArrSend (CRC 校验)`
- ② hex 编码 → `_strSend`
- ③ 写日志 → `WriteErrorLog("发送 报文: " + _strSend)`
- ④ 判断通讯模式：
 - COM (`m_blCom1_Can0=1`) → `RS232.WriteComm(_btArrSend)`
 - CAN (`m_blCom1_Can0=0`) → `Can_Send(_strSend)` → `CAN_ChannelSend([DllImport])`
- ⑤ `WaitTime(10ms)`

10.3 底层 P/Invoke

`CanCmd.CAN_ChannelSend()` 声明在 `Clb_MT_Comm/CanCmd.cs` :

```
[DllImport("CanCmd.dll")]
public static extern UInt32 CAN_ChannelSend(
    UInt32 dwDeviceHandle,
    UInt32 dwChannel,
    ref CAN_DataFrame pSend,
    UInt32 Len);
```

CAN_DataFrame 结构体：8 字节数据 (arryData[8]) + 帧 ID (uID) + 数据长度 (nDataLen) + 发送类型/帧格式标志。

十一、演进路线图

优先级	模块	状态	说明
P0 ✓	基础运动 (前进/后退/停止)	已完成	3 个 SendData 调用 + 连接/断开心跳
P0 ✓	新窗体 + 接口 + 服务层骨架	已完成	Program_New.cs → Frm_Main_C_New 完整 DI 链路
P1	速度控制	待实现	需要模式判断(手动/自动)
P1	距离读取	待实现	需解析 CAN 回传帧编码器位置
P2	光栅臂控制	接口已实现	涉及位置范围参数
P2	TofdHardwareAdapter	空桩	需接入 Tofd_DLL.CLTofd_BJ
P2	CameraAdapter	空桩	需接入 Clb_XmCam_DLL
P3	打标 / 自动扫查	接口已预留	涉及复杂动作序列
远 期	ICScanHardware / ILaserProfiler / IPowerMonitor	待规划	适配器 + 接口均已定义

十二、新增文件清单

#	文件	行数	职责
1	ClassLibrary_Interface/IHardwareInterfaces.cs	142	7 个硬件抽象接口
2	ClassLibrary_Interface/SystemConfig.cs	186	配置集中加载/保存
3	NewInspect.Services/Services/TofdService.cs	153	TOFD 业务编排
4	NewInspect.Services/Services/MotionService.cs	167	运动控制编排
5	NewInspect.Services/Services/CameraService.cs	116	相机管理
6	NewInspect.Services/Services/DataService.cs	95	数据持久化
7	NewInspect.Services/Adapters/HardwareAdapters.cs	372	3 个硬件适配器
8	NewInspect.Services/State/RuntimeState.cs	186	6 个状态类
9	Program_New.cs	125	DI 入口点
10	From/Frm_Main_C_New.cs	244	新主窗体
11	From/Frm_Main_C_New.Designer.cs	225	控件布局定义
新框架总计		2,011	vs 旧框架 36,154 行

代码精简比：新框架 2,011 行 vs 旧框架 (Frm_Main_C + SysInfo + Struct) 36,154 行 = 仅 5.6%。核心原因：新框架不重复实现已有 DLL 逻辑，只做接口适配和业务编排。